

SWF9 文件格式规范

介绍.....	2
SWF 文件格式规范.....	2
SWF 文件头.....	3
SWF 文件结构.....	4
标签格式.....	5
定义型标签和控制型标签.....	5
SWF 文件中标签（Tag）的排序.....	6
字典.....	6
SWF 文件的处理过程.....	7
文件压缩措施.....	7
总结.....	8
第一章 SWF9 的文件格式.....	9
第二章 SWF8 文件格式.....	11
第五章 基本数据类型.....	13
坐标和线.....	13
整数类型和字节顺序.....	13
定点数（Fixed-point numbers）.....	14
浮点数字.....	14
整数编码.....	15
位值.....	16
使用位值.....	17
字符值.....	17
语言代码.....	18
RGB 颜色信息（RGB color record）.....	18
带 Alpha 的 RGBA 颜色信息.....	19
带 Alpha 的 ARGB 颜色信息.....	19
矩形信息.....	19
矩阵信息.....	20
颜色转换信息.....	21
带 alpha 的颜色转换记录.....	22
第六章：显示列表.....	24
剪辑层.....	25
使用显示列表.....	25

SWF9 文件格式说明

作者: 闪刀浪子 <http://www.iflashigame.com> http://hi.baidu.com/mr_ziqiang

原文: swf_file_format_spec_v9.pdf

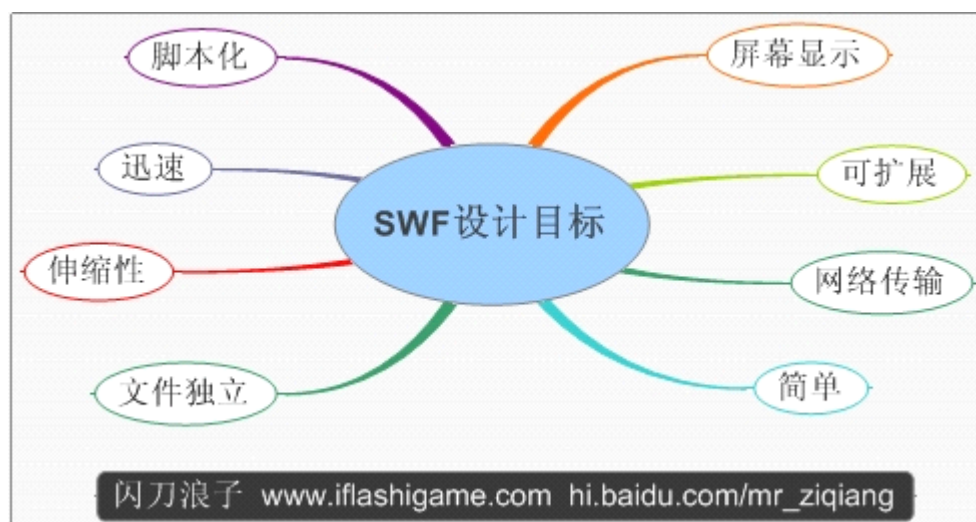
Swf 文件格式规范参考: <http://blog.csdn.net/alenwelkin/archive/2008/02/12/2088719.aspx> 作者: kk

介绍章节其他部分参考: <http://blog.csdn.net/flashvan/archive/2004/09/30/121482.aspx> 作者: AOL

介绍

SWF 文件格式规范

SWF 格式(念"swiff")主要用于在互联网上传递矢量图、文本、视频和声音,且能被 Adobe Flash Player 软件支持。它被设计为一种高效的传输格式,而不是用于图形和图形编辑器的交换格式。以下是它的设计目标:



适合屏幕显示——需要适合屏幕显示同时支持抗锯齿。能够快速渲染各种颜色格式的位图、动画和交互按钮。(ps:比如 ARGB、RGBA 等)

易于扩展——使用标签格式,让他能够很容易保证与 Flash Player 之前的版本兼容同时又能随意扩展新的特性。

方便网络传输——要能够在有限或者不确定的带宽下传输。文件使用压缩来减少尺寸同时支持流式渲染。采用的是二进制格式的标签而不是像 HTML 一样的可读性标签,同时使用了 bit-pack (ps:相当于内置一些编译好的二进制) 和标记位结构 (structures with optional fields) 的技巧来让文件跟小。

(ps: 标记位结构表示用一个字节中的某几个位来作为标记, 比如: 某字节的二进制位:10001010, 最高位的为 1 表示某个标记位 *true*, 如果为 0 表示某个标记位 *false*)

简单——这种格式需要尽量简单以使他的播放器小一点容易实现。同时他还必须尽量少依赖操作系统的特性, 方便跨平台。

文件独立——这种文件的现实不要依赖像字体这样外部的资源

可伸缩性——在有限硬件条件下能够运行, 同时在硬件条件好的情况下能够充分发挥硬件优势。这一点很重要因为计算机有不同的显示分辨率和处理能力。

迅速——高速、高质量的渲染。

脚本实现——这种格式包含各种标签, 标签提供堆栈机解释需要的字节码。字节码支持 ActionScript 语言。Flash Player 提供运行时的 ActionScript 对象模型允许 Flash Player 的图元、服务交互。

SWF 文件使用 .swf 作为扩展名, application/x-shockwave-flash 为 MIME 类型。

SWF 格式经历了几个版本。直到 SWF5 才真正明确了 SWF 标签设置。SWF6 及后面的版本改变很小, 一些新的特性部分或者全部在 ActionScript 脚本层实现。(ps: 底层不怎么变了) 从 SWF9 开始, AS3 使用了新的虚拟机。任何打算使用新特性来生成 swf 文件内容都需要熟悉 Flash Player 公开的 ActionScript 对象模型 (ActionScript object model)。更多的信息请参考 “ActionScript3.0 编程”, “ActionScript 3.0 语言参考” 和 Adobe Action Script 虚拟机 2 的概览。(www.adobe.com/go/avm2overview/)

SWF 文件头

所有的 SWF 文件都由文件头开始。数据类型定义请阅读后面的 “基本数据类型” 章节。
(类型中 U 表示无符号, I 表示整型。下面的 UI8 表示占 8 位二进制的无符号整型)

SWF 文件头		
字段	类型	说明
标志	UI8	标志字节: 如果是 “F” 表示未压缩, 如果是 “C” 表示压缩 (SWF6 之后才有)
标志	UI8	始终为 “W”
标志	UI8	始终为 “S”
版本号	UI8	一个字节的版本号 (如 0x06 表示 SWF6)
文件长度	UI32	完成文件的长度有多少字节
显示尺寸	RECT 结构	以 twips 为单位的显示尺寸, 20twips=1px, Rect 结构请阅读后面

		的基本数据类型章节。
帧速	UI16	每秒的帧速，通过一个 8.8 小数方式显示。(整数和小数各占 8 位)
帧总数	UI16	文件中帧的总数。

文件头都是由三个字节开始的 0x46, 0x57, 0x53 (“FWS”); 或者 0x43, 0x57, 0x53 (“CWS”)。FWS 表示文件没有压缩, CWS 表示第 8 个字节后面是经过 ZLIB 开放标准压缩的(文件长度字段后面)。SWF6 之后的版本才开始使用压缩。

版本号占一个字节紧跟在标志之后。版本数字不是一个 ASCII 字符而是一个 8 位的数字。例如 SWF4, 版本号字节为 0x04, 而不是 ASCII 字符“4”(0x35)。

文件长度字段记录 SWF 文件的总长度, 包括文件头。对于未压缩的 swf 文件(标志位 FWS), 文件长度字段的数值跟该文件长度一致。而对于压缩的 swf 文件(标志为 CWS), 文件长度字段的长度表示第 8 个字节之后的数据解压缩之后的长度。(ZLIB 标准)

显示尺寸字段定义了屏幕显示的宽度和高度。这个字段使用一个 RECT 结构来存储(一种可变长度的存储结构, 详细信息看基本数据结构中对于 RECT 的介绍), 这表示它的占位长度会根据 bits 的描述来随时改变。显示尺寸字段的 RECT 结构的 Xmin 和 Ymin 通常为 0; Xmax 和 Ymax 定义宽度和高度。

帧速字段定义了每秒的播放速率。这个速率无法绝对保证, 比如 CPU 忙的时候或者慢的时候。

帧总数定义了 SWF 文件的总帧数。(ps:注意到没有? 只有 16 位, 那么帧总数的极限只能是 65535 帧了)

SWF 文件结构

文件头之后是一系列的标记的数据块(ps: 一般成为标签 Tag)。所有的标签(Tag)都使用公用的格式, 所以程序在分析 SWF 文件的时候都可以跳过一些不明确的数据块。数据块里面的数据可以定位到内部的偏移位置, 但是不能定位到其他块中的偏移位置。(ps:主要是保证 Tag 间相互独立, 记得 flash 的设计灵感是来自乐高积木的。不同的 Tag 相当于不同的积木块。)这样的方式使我们可以处理 SWF 文件的时候通过工具移除、插入、编辑标签。

SWF 文件结构图



FileAttributes 标签在 SWF8 之后才需要。

标签格式

每一个标签的开始都是标签的类型和长度。标签头可以是短标签头类型也可以是长标签头类型。短标签头类型用于数据在 62 字节之内的标签。如果标签的数据长度不超过 62 个字节使用短标签。而长标签头类型可以用于数据在 4G 之内的任何标签。

标签头记录（短类型）		
字段	类型	说明
TagCodeAndLength	UI16	高 10 位表示标签类型 低 6 位表示标签长度

注意 事项	TagCodeAndLength 字段是一个两个字节的字，而不是一个 10 位的字段跟随一个 6 位字段。SWF 使用低位在前（little-endian）造成上述两种情况是不同的。 (ps: 如果 TagCodeAndLength 字段的十六进制是 43 02 那么实际上的值是 0234 二进制为 00000010 01000011 前 10 位的值是 9 表示这是一个 SetBackgroundColor 标签)
----------	---

标签的长度不包括标签头记录自己的长度，仅表示其后面数据的长度。

如果标签为 63 字节或者更长，它将存储为一个长标签头的类型。长标签头可以看成是一个标签长度为 0x3f 的短标签跟着一个 32 位的长度。(ps:这表示只要 TagCodeAndLength 中的后 6 位的值为 0x3f，那么肯定是一个长标签头类型。)

标签头记录（短类型）		
字段	类型	说明
TagCodeAndLength	UI16	高 10 位表示标签类型 低 6 位始终为 0x3F
Length	SI32	标签的长度

定义型标签和控制型标签

Swf 文件有两类标签：定义型和控制型

定义型标签定义 SWF 文件的内容如 Shapes、文本、位图、声音等等。每一个标签会为定义的内容分配一个唯一的编号称做角色编号（character ID）。之后 Flash Player 将角色（character）存储在称作字典（dictionary）的仓库。定义型标签本身不产生渲染。

控制型标签创建和操作已渲染的字典中角色的实例，同时控制文件流程。

SWF 文件中标签（Tag）的排序

一般来讲，SWF 中的标签（Tag）可以随意排序。但是必须遵循以下规则：

- 如果是 SWF8 之后的版本则 FileAttributes 标签必须为第一个标签
- 标签仅依赖之前的标签，而不依赖后面的标签。（ps:我的理解是后面需要用到的内容始终会在前面先定义，跟下一条控制型标签的规则类似）
- 角色的定义标签必须出现在引用他的控制型标签之前。
- 声音流标签必须有序。无序的声音标签会导致播放时也无序。
- 结束标签始终在 SWF 文件的最后

字典

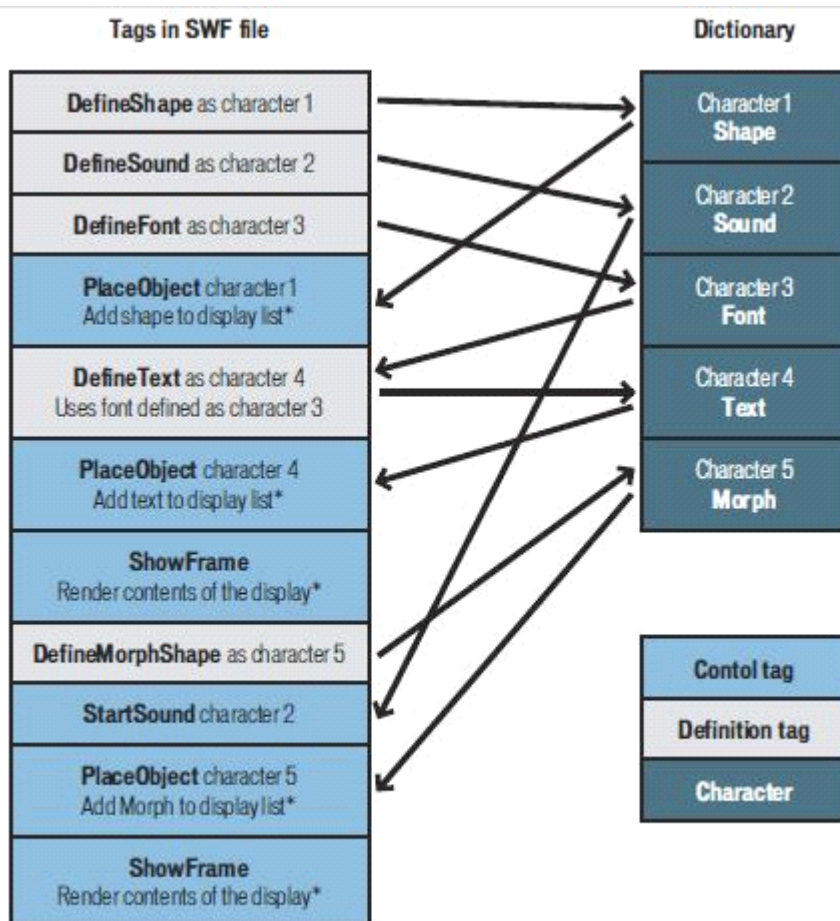
字典是定义的角色存储的仓库，给控制型标签使用。字典的创建和使用的流程是这样的：

- 1、定义型标签定义内容，像形体、字体、位图或者声音。
- 2、定义型标签给内容分配唯一的角色编号（CharacterID）
- 3、根据角色编号把内容存储到字典中。
- 4、控制型标签使用角色编号在字典中检索到内容，并对内容执行相关动作，比如显示一个形体（Shape）或者播放一段声音。

每一个定义型标签只能指定一个唯一的编号，不允许相同。举一个象征性的例子：第一个角色编号为 1，第二个角色编号为 2，以此类推。数字 0 有特殊含义表示一个空的角色。

并不是只有控制型标签可以引用字典。定义型标签也可以使用字典中已经存在的角色来定义更加复杂的角色。例如：DefineButton 和 DefineSprite 标签都是通过其他的角色来定义他们的内容（PS:相当于容器里面放内容）。DefineText 标签可以通过字体角色来为文本选择不同的字体。

下图是一个典型的定义型标签、控制型标签和字典的关系：

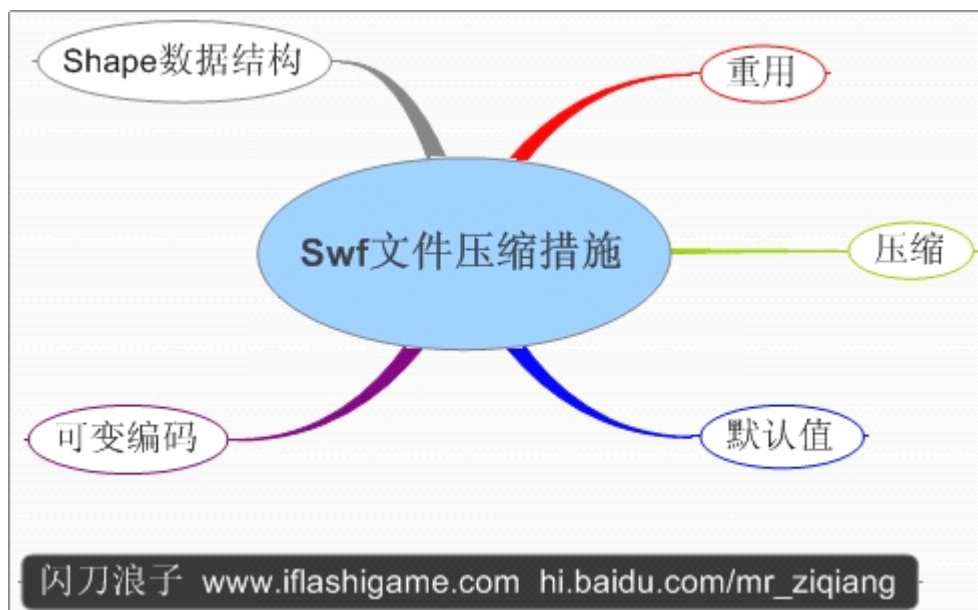


SWF 文件的处理过程

Flash Player 会处理所有的标签直到遇到 ShowFrame 标签。此时，显示列表被复制到屏幕，然后播放器继续处理直到下一帧需要显示。第一帧的内容累计了第一个 ShowFrame 标签之前所有控制型标签的执行结果，第二帧的内容累计从文件开始到第二个 ShowFrame 标签的所有控制标签的执行效果，以此类推。

文件压缩措施

SWF 文件需要在网络上频繁传输，所以自然是越紧凑越好。下面的技巧就是被用来实现这一点：



重用——角色字典的结构可以让非常容易实现这一点。例如，形状、按钮、声音、字体或者位图只在文件中存一次，却能多次引用。

压缩——形状通过一种有效的偏差编码方式来压缩。通常直线开始的坐标被认为是上一条线的终点。距离通常是相对于上一个点来表达的。

默认值——有一些结构，像 `matrixes` 和 `color transform` 有一些公用字段用的比其他多。比如一个矩阵，最常使用的就是转换。缩放和旋转比较少用。所以，如果缩放没有定义，它默认为100%，如果旋转没有定义，则默认不旋转。通过这种默认值的方式来让文件更小。

变化编码——作为一种规则，SWF 文件只存储状态之前的变化。它由 Shape 数据结构和显示列表中使用的放置-移动-移除模块体现。

Shape 数据结构——Shape 数据结构使用了一种特别的结构。使形体的大小最小化，同时在屏幕上渲染抗锯齿效果非常有效。

总结

SWF 文件是由一个文件头，和跟在后面的一系列的标签组成。标签有定义型标签和控制型标签两类。定义型标签把对象定义为角色存储在字典里，控制型标签操作这些角色并控制影片的流程。

第一章 SWF9 的文件格式

本章描述 SWF9 文件格式规范中的新特性和标签。

ActionScript 3.0

SWF9 引入了新的 AS3 语言和虚拟机。为了明确一个 SWF 是否需要用到新的虚拟机，FileAttributes 标签加了一个 ActionScript3 的字段。本章中描述的东西只有在这个 ActionScript3 字段为 1 时有效。

AS3 字节码包含在新的 DoABC 标签中。关于 AS3 字节码的详细资料可以在 Adobe 虚拟机 2 (AVM2) 的概览中找到。(www.adobe.com/go/avm2overview/)

ActionScript2 字节码

AS2 字节码在 AS3 的 SWF 文件中不支持。包括 DoAction 和 DoInitAction 标签，以及 DefineButton2 标签中的 Action 部分。这些标签中的字节码将被忽略。

类链接 (Class linkage)

很多新的标签已经被添加或修改，用来支持和扩展 AS3 的类和运行实例之间的链接。SymbolClass 标签用来链接 SWF 文件中的元件 (Symbol) 和 AS3 类。SymbolClass 是 AS3 特有的，相当于 ExportAssets 标签。

PlaceObject3 标签现在支持了 PlaceFlagHasClassName 字段。这个字段表示指定了一个类名，作为对象的类型去放置。同时还支持 PlaceFlagHasImage 字段。表示将从一个 characterID 来创建一个位图或者基于一个类名从其他的 SWF 文件中加载来创建位图。

DefineEditText 标签现在支持了 HasFontClass 字段。表示指定一个类名作为字体类从其他的 SWF 中加载。

添加了 StartSound2 标签用来跟载入的 SWF 文件的声音一起使用，同时包含了声音类的名称。

二进制数据

DefineBinaryData 标签允许在 SWF 文件中植入任何二进制数据。DefineBinaryData 用来跟 SymbolClass 标签联合使用。SymbolClass 标签能用来将一个 DefineBinaryData 标签和一个 AS3 类定义联系起来，这个类必须为 ByteArray 的子类。当一个类被实例化，他将自动拥有二进制数据源的内容。

文本和字体

因为各种原因，静态文本域能作为动态文本域导出到 SWF。为了确定这样的文本域，DefineEditText 标签添加了 WasStatic 字段。这将允许 AS3 把这样的文本域作为静态文本来看待。同时为了包含植入在 SWF 文件中的字体的版本信息添加了 DefineFontName 标签。

场景支持

新的 DefineSceneAndFrameLabelData 标签描述了影片剪辑中发现的场景和帧。多场景只支持主时间轴，对于其他的影片剪辑只连通其他帧导出单一的场景。

第二章 SWF8 文件格式

本章介绍 SWF8 文件规范中的新特性和标签

位图滤镜

SWF8 在运行时引入了丰富的图像效果而不影响文件的大小。新的位图滤镜包括模糊、阴影、发光、导角、渐变发光、渐变导角、颜色矩阵和其他的能应用在显示对象之上的，PlaceObject3 标签中 FILTERLIST 和 FILTER 记录的效果。

混合模式

SWF8 以后的版本，使用混合模式将显示对象的图片或者图片按钮和下层进行效果合成成为了可能。通过 PlaceObject3 标签和 Button record，可以在正常和透明混合之间任选一种混合模式。

位图缓存

回放性能能够通过位图缓存来进行优化。通过 PlaceObject3 标签可以指定一个静态的影片剪辑或者一个按钮原件在运行时作为位图缓存起来。Flash 播放器在内部缓存显示对象而不用频繁重新绘制图像，这种方式能够为某些特定需求下的回放性能提供重要的改进方式。

加强轮廓

DefineShape4 和 DefineMorphShape2 标签中的 LINESTYLE2 和 MORPHLINESTYLE2 的记录信息，在以前一版本的 SWF 文件的基础上允许使用新的接合、端点填充轮廓。当时较早的线型记录只允许圆角结合和圆角端点，而 LINESTYLE2 和 MORPHLINESTYLE2 还能支持尖角、斜角接合和方形端点、无端点。同时新的线型也包括了纠正直线和竖线模糊的笔触提示。

加强渐变

SWF8 加强了渐变的控制。FILLSTYLE 和新的 FOCALGRADIENT 记录增加了新的放射渐变类型并允许设置焦点。此外 GRADIENT 记录现在可以有多达 15 个控制点，SWF7 以前只有 8 个控制点

高级文本渲染

DefineFont3, DefineFontAlignZones 和 CSMTTextSettings 标签为 Flash 播放器带来了新的抗锯齿技术。高级文本渲染引擎能够完成高质量的字体渲染，尤其在小字号的时候也能看起来很舒服。

9 切片缩放

DefineScalingGrid 标签引入了 9 切片的概念，允许 Sprite 和按钮保证完整性的情况下进行更好的局部缩放。

高保真 On2 VP6

SWF6 以后，Flash Player8 支持就高保真 On2 VP6 流式格式——一种先进的视频压缩算

法。另外, Flash Player8增加了对 Alpha 通道的支持在视频中能够使用透明效果。

屏幕视频 V2流式编码器(Screen Video V2 bitstream codec)

SWF6支持屏幕视频 V2流式格式,它是屏幕视频流式格式(Screen video bitstream format)的扩展,Flash Player8以后的版本支持这种格式。这种格式使用了特殊的技术来减少数据量来描述屏幕内容。

SWF 文件特征和元数据

从 SWF8开始, SWF 文件必须用新的 FileAttributes 标签作为第一个标签。这个标签标识 SWF 文件自己的特征。

在 SWF8中, FileAttributes 标签有两个目的。此标签表明 SWF 文件中是否包含 SymbolClass 标签,同时使用像 title 和 description 这样的属性来为外部处理进行描述,比如提供给搜索引擎。此外,对于本地播放的 SWF 文件,FileAttributes 指定了 Flash Player 是本地访问还是网络访问。

ImportAssets2

SWF8以后的版本 ImportAsset2标签替代了 ImportAssets 标签。具有 ImportAssets 的功能。

第五章 基本数据类型

这部分描述 SWF 文件中用于拼凑复杂数据结构的基本数据类型。SWF 中所有的其他类型都是基于这些基础的类型

坐标和缇

SWF9文件格式中所有的 x-y 坐标信息都被存为整数，计量单位称作缇（twips）。在 SWF 中，1缇是1/20个逻辑像素。当 swf 文件显示为100%时，逻辑像素等同屏幕像素不做缩放。

例如，一个宽高位800twips 400twips 的矩形渲染时为40px 20px。含有小数的像素尺寸会进行抗锯齿显示为一个近似尺寸。如果是一个790twips*390twips（39.5px*19.5）的矩形显示的出来边缘会有一点模糊。

缇是大小和精度之间的一个好的折中方案。在处理小数坐标的时候，他为缩放和精确放置提供了亚像素精度。

坐标方面 SWF 文件使用传统的屏幕坐标轴。X 表示水平方向从左边的最小值到右边的最大值。Y 表示垂直方向上面的最小值到下面的最大值。

整数类型和字节顺序

SWF 文件使用8位、16位、32位、64位有符号或者无符号的整数类型。所有存储在 SWF 文件中的的整数值都采用低位在前（little-endian）的方式排序：低字节排列在前，高字节排列在后，与 Intel X86体系采用相同的方式。字节中的位排序使用高位在前（big-endian）的方式。高位在前，地位在后。

例如：

一个32位的值0x456e7120，在 SWF 文件中存储为： 20 71 6e 45.

一个16位的值0xe712，在 SWF 文件中存储为： 12 e7

所有的整数类型必须字节对齐。也就是说，整数值的第一位（bit）在 SWF 文件中必须存储在字节的第一位(bit)。

符号整型使用传统的2补码位形式(2's complement bit patterns). 这是大多数现代计算机上采用的符号整型方式。在2补码系统中，负数的第一个位为1，0和正数第一位为0。对于一个负数，-n 描述成这个数的正数 n-1取反。

整数类型	
类型	解释
SI8	8位的符号整型
SI16	16位的符号整型
SI32	32位的符号整型
SI8[n]	带符号的8位数组, n 是元素的个数
SI16[n]	带符号的16位数组, n 是元素的个数
UI8	8位的无符号整型
UI16	16位的无符号整型
UI32	32位的无符号整型
UI8[n]	无符号的8位数组, n 表示元素数量
UI16[n]	无符号的16位数组, n 表示元素数量
UI24[n]	无符号的24位数组, n 表示元素数量
UI32[n]	无符号的32位数组, n 表示元素数量
UI64[n]	无符号的64位数组, n 表示元素数量

定点数 (Fixed-point numbers)

SWF 文件格式支持两种定点数：32位和16位。

32为定点数是16.16方式。高16位表示小数点之前的部分，低16位表示小数点之后的部分。每部分的长度也像32位整型一样存储（低位在前排序，字节对齐）

例如：7.5这样一个值相当于0x0007.8000，他将被存储为00 80 07 00。（ps:0.5为啥会存为80, 这个是规矩： $5=8*16^{-1}+0*16^{-2}$ ）

SWF8之后支持16位8.8带符号的定点数字。高8为整数部分，低8为小数部分。

定点类型	
类型	解释
FIXED	32位 16.16定点数字
SI16	16位8.8定点数字

浮点数字

SWFF8以后支持 IEEE 标准754 浮点类型的使用，下面三种浮点类型

浮点类型	
类型	解释
FLOAT16	半精度浮点数
FLOAT	单精度浮点数
DOUBLE	双精度浮点数

在 FLOAT16 中改变了标准中分配给尾数和指数的位数：

- 1、1位表示符号
- 2、5位表示5位是分配给指数部分，实际的指数是5位数表示的数和16的差值
- 3、10位表示位数

整数编码

Swf9 以后支持变长字节的整数编码，目前支持一种类型。

浮点类型	
类型	解释
EncodedU32	可变长度编码的32位无符号整型

32位无符号整型的值通过一个可变长度字节进行存储。所有 Encoded32's 值的编码都依赖于他的值的大小。（值越大也就需要多一点空间，1-5个字节）。这种编码方式是这样来定义的：如果一个字节的最高位进行了设置，那么表示下一个字节也是值的一部分。每一个字节中只有7位是值，最高位如果有效则使用下一个字节，或者已经是最后一个字节了。

EncodedU32的分析算法如下：

```
int GetEncodedU32(unsigned char*& pos)
{
    int result = pos[0];
    if (!(result & 0x00000080))
    {
        pos++;
        return result;
    }
    result = (result & 0x0000007f) | pos[1]<<7;
    if (!(result & 0x00004000))
    {
        pos += 2;
        return result;
    }
    result = (result & 0x00003fff) | pos[2]<<14;
    if (!(result & 0x00200000))
    {
        pos += 3;
        return result;
    }
    result = (result & 0x001ffff) | pos[3]<<21;
    if (!(result & 0x10000000))
    {
        pos += 4;
        return result;
    }
}
```



```
}
result = (result & 0xffffffff) | pos[4]<<28;
pos += 5;
return result;
}
```

位值

位值是可变长度位字段的表现方式，可以描述三种数字类型

- 1、无符号整型
- 2、符号整型
- 3、带符号的16.16定点值

位值没有字节对齐，其他的类型（像 UI8和 UI16）要求字节对齐。如果位值后面跟着一个字节对齐类型，那么包含位值的最后位用0补齐。

下面的例子是一个64位的流，表示了9个变位长的值，跟着一个 UI16值。

```
Byte1---Byte2---Byte3---Byte4---Byte5---Byte6---Byte7---Byte8---
010110101001001001011100100011010111001100100000100110010101101
BV1---BV2---BV3---BV4---BV5-----BV6BV7---BV8BV9---pad-U16-----
```

上面的字节流中开始是占6个位的 BV1，下来是占5个位的 BV2，BV2 跨越了两个字节 Byte1 和 Byte2……BV9跟着一个 U16所以需要用0来补齐。

位值	
类型	解释
SB[nBits]	带符号的位值（nBits 是用于存储值所需要的位数）
UB[nBits]	无符号位值（nBits 是用于存储值所需要的位数）
FB[nBits]	带符号的定点位值（nBits 是用于存储值所需要的位数）

当一个无符号的位值扩充到一个字（双字节），左侧的位全部填充0。当一个带符号的位值扩充到一个字，高位复制到左侧。这种扩充叫做符号扩充。例如一个4个位的无符号位值 UB[4]=1110扩充到一个16位的值就是这样：0000000000001110=14。同样，一个带符号的值 SB[4]=1110将被扩充为这样：111111111111110=-2。符号位的值是相似的，但是必须考虑符号本身的占位。带符号的值35描述为这样 SB[7]0100011。高位的0是必须的。如果高位是1就是一个负数。

定点位值是一个32位的16.16的带符号定点数字。即高16位是小数点前面的部分，低16位是小数点后面的部分。定点位值转符号位值也是一样（同上面的扩充方式一样），只是解释有些不同。例如，一个19位的带符号的位值0x30000的十进制是196608。19位的定点位值 0x30000却是3.0。这种3.16的格式比16.16的格式更加有效（ps：我的理解这是说19位的定点位置实际上不用32个位只用了19个位就搞定了）

使用位值

位置用尽可能少的位数来存储值。大部分的位值字段都使用固定数字的位，也有一些使用可变的数字位。但是，在所有类似的案例里，同样的结构里，位数被用来明确不同的字段中的状态。在这些变长的案例和生成 SWF 的应用中必须确定位所需的最小值，用于描述指定的实际值。

对于带符号的位值，如果被编码的位数是确定的，必须保留扩展位0；否则符号扩展会将这个值变为一个负数。

RECT 结构就是一个变长位置的例子

RECT		
字段	类型	说明
Nbits	UB[5]	每一个 RECT 值所需要的位的长度
Xmin	SB[Nbits]	X 的最小值
Xmax	SB[Nbits]	X 的最大值
Ymin	SB[Nbits]	Y 的最小值
Ymax	SB[Nbits]	Y 的最大值

Nbits 字段描述了用来存储 Xmin Xmax Ymin Ymax 坐标值所需要的位长度。假定一个矩形的坐标如下：

Xmin = 127 decimal = 1111111 binary

Xmax = 260 decimal = 100000100 binary

Ymin = 15 decimal = 1111 binary

Ymax = 514 decimal = 1000000010 binary

Nbits 是求得的坐标需要的位数最多的值。在这个例子里，值514是需要位数最多的需要11位。他们将像下表一样存储：

RECT		
字段	类型	说明
Nbits	UB[5]=1101	位长为 11
Xmin	SB[11]=00001111111	127 缋
Xmax	SB[11]=00100000100	260 缋
Ymin	SB[11]=00000001111	15 缋
Ymax	SB[11]=01000000010	514 缋

字符值

字符值描述为一个空值结束的字符串。字符值是一个以空字符结束的序列。

STRING		
字段	类型	说明
String	UI8[zero or more]	不为空的字符串数据

StringEnd	UI8	标记字符串的结尾，永远为 0
-----------	-----	----------------

在 swf5 及以前的版本中，字符串类型不是使用 ANSI 编码(ASCII 扩展集)就是采用 shift-JIS(一种日本编码)。并且我们没有办法确定它到底采用的是哪种编码方式，具体如何解码是由播放的时候，播放器根据本地的情况自动检测的。这意味着 SWF5 或更早的文件仅能使用 ANSI 或 shift-JIS 编码文本内容，并且创建时，目标观众必须是已知的一否则容易混淆文本结果，这样就很容易出现错误。

在 swf6 及以后版本中，字符串的值经常使用 Unicode UTF-8 编码。它是一种多字节的编码方式，它对字符编码使用 1-4 个字节。UTF-8 编码是 ASCII 的一个超集，在 0-127 之间的字符排列顺序是按照 ASCII 中的顺序来进行排列的，并且 0-127 之间的字符都只要一个字节描述。UTF-8 保证除了空字符以外的所有字符都使用一个以上的字节来表示，并且这些字节里面没有一个字节的值为 0。这就保证了在一个 UTF-8 字符串里面不会出现不希望出现的空字符。

语言代码

语言代码表示文字的母语，他跟 swf 文件中的字体规范有关。(ps:语言代码不是确定文本的编码，而代表母语)

LANGCODE		
字段	类型	说明
LanguageCode	UI8	语言代码

Flash 播放器使用语言代码来为动态文本确定断行规则，以及当指定设备字体不存在时选择什么默认字体。语言代码的其他用途可能以后才能知道。

如果语言代码为 0 表示没有设置语言。Flash 播放器在运行时根据本地的系统来决定。

Flash 播放器现在可以使用以下语言：

拉丁语

日语

韩语

简体中文

繁体中文

RGB 颜色信息 (RGB color record)

用 24 位的红、绿、蓝值描述颜色

RGB		
字段	类型	说明
Red	UI8	红色值

Green	UI8	绿色值
Blue	UI8	蓝色值

带 Alpha 的 RGBA 颜色信息

用32位的红、绿、蓝、透明值描述颜色。RGBA 颜色用 alpha=255表示不透明，alpha=0表示透明。中间值表示半透明。

RGBA		
字段	类型	说明
Red	UI8	红色值
Green	UI8	绿色值
Blue	UI8	蓝色值
Alpha	UI8	透明值

带 Alpha 的 ARGB 颜色信息

ARGB 和 RGBA 类似，只是第一位表示 alpha 值。

ARGB		
字段	类型	说明
Alpha	UI8	透明值
Red	UI8	红色值
Green	UI8	绿色值
Blue	UI8	蓝色值

矩形信息

矩形值通过定义 x、y 的最小值和最大值描述矩形。RECT 信息必须字节对齐。

RECT		
字段	类型	说明
Nbits	UB[5]	每一个 RECT 值所需要的位的长度
Xmin	SB[Nbits]	X 的最小值
Xmax	SB[Nbits]	X 的最大值
Ymin	SB[Nbits]	Y 的最小值
Ymax	SB[Nbits]	Y 的最大值

矩阵信息

矩阵信息描述一个标准2x3的用于普通二维图形的变换矩阵。它一般用于描述一个图形对象的缩放、旋转和转换。这个矩阵块必须字节对齐。

MATRIX		
字段	类型	说明
HasScale	UB[1]	1表示有缩放
NScaleBits	If HasScale = 1, UB[5]	每个缩放值字段需要的位数
ScaleX	If HasScale = 1, FB[NScaleBits]	X 缩放值
ScaleY	If HasScale = 1, FB[NScaleBits]	Y 缩放值
HasRotate	UB[1]	1表示有旋转或者倾斜
NRotateBits	If HasRotate = 1, UB[5]	每一个旋转至需要的位数
RotateSkew0	If HasRotate = 1, FB[NRotateBits]	第一个旋转和倾斜的值
RotateSkew1	If HasRotate = 1, FB[NRotateBits]	第二个旋转和倾斜的值
NTranslateBits	UB[5]	每一个转换值需要的位数
TranslateX	SB[NTranslateBits]	X 转换值单位为缇
TranslateY	SB[NTranslateBits]	Y 转换值单位为缇

ScaleX ScaleY RotateSkew0 RotateSkew1 字段存储为一个16.16的定点值。TranslateX 和 TranslateY 使用缇作为单位存储为带符号的值。

如果矩阵信息的公共信息做了优化，在没有其他值的情况下值记录了转换信息（PS：前文中 SWF 压缩策略中已经提到过了）。在这里如果 HasRotate 和 HasScale 标记为0，那么 matrix 值包含 Translate X 和 TranslateY 的字段。

MATRIX 字段映射为2x3的矩阵如下：

ScaleX	RotateSkew0
RotateSkew1	ScaleY
TranslateX	TranslateY

对于任何一个坐标 (x,y)，转换后的坐标 (x',y') 计算结果如下：

$$x' = x * \text{ScaleX} + y * \text{RotateSkew1} + \text{TranslateX}$$

$$y' = x * \text{RotateSkew0} + y * \text{ScaleY} + \text{TranslateY}$$

下表中描述了每一类操作，有多少矩阵成员被使用。

	ScaleX	RotateSkew0	RotateSkew1	ScaleY
旋转操作	余弦	正弦	反正弦	余弦
缩放操作	水平缩放分量	无	无	垂直缩放分量
剪切操作	无	水平比例常量	垂直比例常量	无
镜像操作	水平镜像分量	无	无	垂直镜像分量

颜色转换信息

CXFORM 信息定义了一个简单的转换，能应用到一个图形对象的颜色空间。有如下两种转换方式：

- 1、乘法转换
- 2、加法转换

乘法转换用一个8.8的定点乘法因子（multiplication term）乘以红、绿、蓝的分量。定点值1.0表示0x100或者十进制256。

对于任何颜色（R，G，B），转换后的颜色（R'，G'，B'）的计算如下：

$$R' = (R * \text{RedMultTerm}) / 256$$

$$G' = (G * \text{GreenMultTerm}) / 256$$

$$B' = (B * \text{BlueMultTerm}) / 256$$

加法转换为正在显示的对象加一个因子（正或者负）到红、绿、蓝分量。如果值大于255，那么结果是保持255。如果值小于0，则结果保持0。

对于任何颜色（R，G，B），转换后的颜色（R'，G'，B'）的计算如下：

$$R' = \max(0, \min(R + \text{RedAddTerm}, 255))$$

$$G' = \max(0, \min(G + \text{GreenAddTerm}, 255))$$

$$B' = \max(0, \min(B + \text{BlueAddTerm}, 255))$$

加法和乘法转换可以像下面一样组合。乘法操作优先：

$$R' = \max(0, \min(((R * \text{RedMultTerm}) / 256) + \text{RedAddTerm}, 255))$$

$$G' = \max(0, \min(((G * \text{GreenMultTerm}) / 256) + \text{GreenAddTerm}, 255))$$

$$B' = \max(0, \min(((B * \text{BlueMultTerm}) / 256) + \text{BlueAddTerm}, 255))$$

CXFORM 记录必须字节对齐

CXFORM		
字段	类型	说明
HasAddTerms	UB[1]	1表示有加法转换
HasMultTerms	UB[1]	1表示有乘法转换
Nbits	UB[4]	每一个值字段的位数
RedMultTerm	If HasMultTerms = 1,SB[Nbits]	红色乘法值
GreenMultTerm	If HasMultTerms = 1,SB[Nbits]	绿色乘法值
BlueMultTerm	If HasMultTerms = 1,SB[Nbits]	蓝色乘法值
RedAddTerm	If HasAddTerms = 1,SB[Nbits]	红色加法值
GreenAddTerm	If HasAddTerms = 1,SB[Nbits]	绿色加法值
BlueAddTerm	If HasAddTerms = 1,SB[Nbits]	蓝色加法值

带 alpha 的颜色转换记录

CXFORMWITHALPHA 记录扩展了 CXFORM 的功能，允许颜色转换时用到 alpha 通道和红、绿、蓝通道。

有以下两种类型的转换

- 1、乘法转换
- 2、加法转换

乘法转换用一个8.8的定点值乘以红、绿、蓝、alpha 分量。定点值1.0表示0x100或者十进制256。所以乘法操作后的结果可以被256整除。

对于任何颜色 (R, G, B, A)，转换后的颜色 (R', G', B', A') 的计算如下：

$$R' = (R * \text{RedMultTerm}) / 256$$

$$G' = (G * \text{GreenMultTerm}) / 256$$

$$B' = (B * \text{BlueMultTerm}) / 256$$

$$A' = (A * \text{AlphaMultTerm}) / 256$$

CXFORMWITHALPHA 记录经常通过为 alpha 通道乘以一个0-256之间的值，来使对象部分透明。

加法转换为正在显示的对象的红、绿、蓝、透明分量加一个固定值(正数或者负数)。值不大于255，不小于0。

对于任何颜色 (R, G, B, A)，转换后的颜色 (R', G', B', A') 的计算如下：

$$R' = \max(0, \min(R + \text{RedAddTerm}, 255))$$

$$G' = \max(0, \min(G + \text{GreenAddTerm}, 255))$$

$$B' = \max(0, \min(B + \text{BlueAddTerm}, 255))$$

$$A' = \max(0, \min(A + \text{AlphaAddTerm}, 255))$$

加法和乘法转换可以像下面一样组合。乘法操作优先：

$$R' = \max(0, \min(((R * \text{RedMultTerm}) / 256) + \text{RedAddTerm}, 255))$$

$$G' = \max(0, \min(((G * \text{GreenMultTerm}) / 256) + \text{GreenAddTerm}, 255))$$

$$B' = \max(0, \min(((B * \text{BlueMultTerm}) / 256) + \text{BlueAddTerm}, 255))$$

$$A' = \max(0, \min(((A * \text{AlphaMultTerm}) / 256) + \text{AlphaAddTerm}, 255))$$

与 CXFORM 一样，CXFORMWITHALPHA 也是字节对齐

CXFORMWITHALPHA		
字段	类型	说明
HasAddTerms	UB[1]	1表示有加法转换
HasMultTerms	UB[1]	1表示有乘法转换
Nbits	UB[4]	每一个值字段的位数
RedMultTerm	If HasMultTerms = 1, SB[Nbits]	红色乘法值
GreenMultTerm	If HasMultTerms = 1, SB[Nbits]	绿色乘法值
BlueMultTerm	If HasMultTerms = 1, SB[Nbits]	蓝色乘法值
AlphaMultTerm	If HasMultTerms = 1, SB[Nbits]	Alpha 乘法值
RedAddTerm	If HasAddTerms = 1, SB[Nbits]	红色加法值
GreenAddTerm	If HasAddTerms = 1, SB[Nbits]	绿色加法值

BlueAddTerm	If HasAddTerms = 1,SB[Nbits]	蓝色加法值
AlphaAddTerm	If HasAddTerms = 1,SB[Nbits]	Alpha 加法值

第六章：显示列表

显示 SWF 文件的一个帧有三个舞台处理：

- 1、用 DefineShape, DefineSprite 等定义型标签定义好对象。每一个对象给一个唯一的 ID 称为角色，他们保存在字典里。
- 2、从字典中复制选中的角色放置到显示列表。显示列表是将要显示在下一帧的角色的列表。
- 3、完成之后，把显示列表的内容用 ShowFrame 显示在屏幕上。

显示对象的每一个角色都分配到一个深度值。深度值确定角色的堆栈顺序。深度值低的角色在深度值高的对象的下面。深度值为1的在堆栈的最底下。一个角色可以多次出现在显示列表，但深度不同。一个深度只有一个角色。

在 SWF1和2中，显示列表是平面化的列表，任何时候进入到场景都是如此。在 SWF3 与后面的版本中，显示列表是一个分级列表。显示的元素能够有自己的子元素显示列表。更多的信息查阅 DefineSprite 标签。

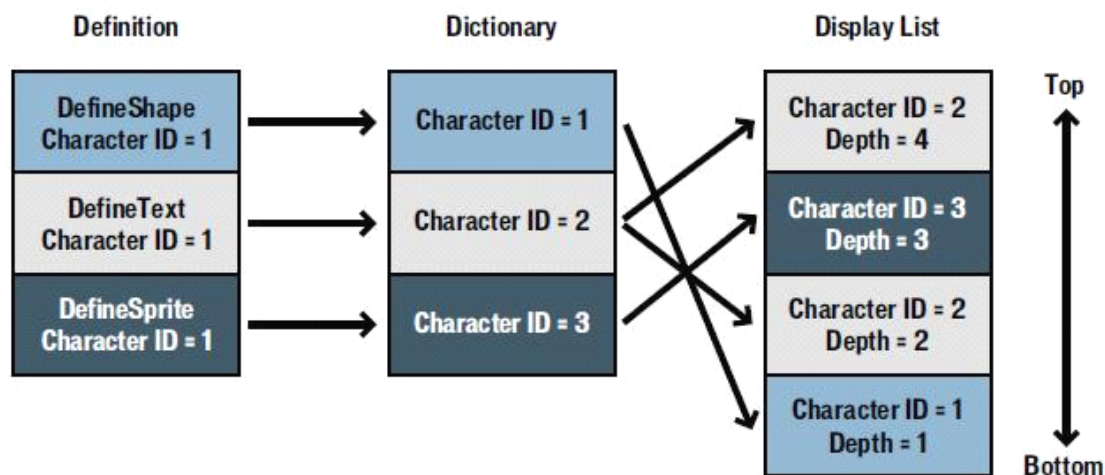
以下6个标签用于控制显示列表：

- 1、PlaceObject——添加一个角色到显示列表
- 2、PlaceObject2&PlaceObject3——添加一个角色到现实列表或者修改指定角色的深度。
- 3、RemoveObject——从显示列表移除一个指定的角色
- 4、RemoveObject2——从指定的深度移除一个角色
- 5、ShowFrame——渲染显示列表的内容到显示器

注意：旧的标签 PlaceObject 和 RemoveObject 很少在 SWF3以后版本中使用了。

下图举了一个现实处理的例子。

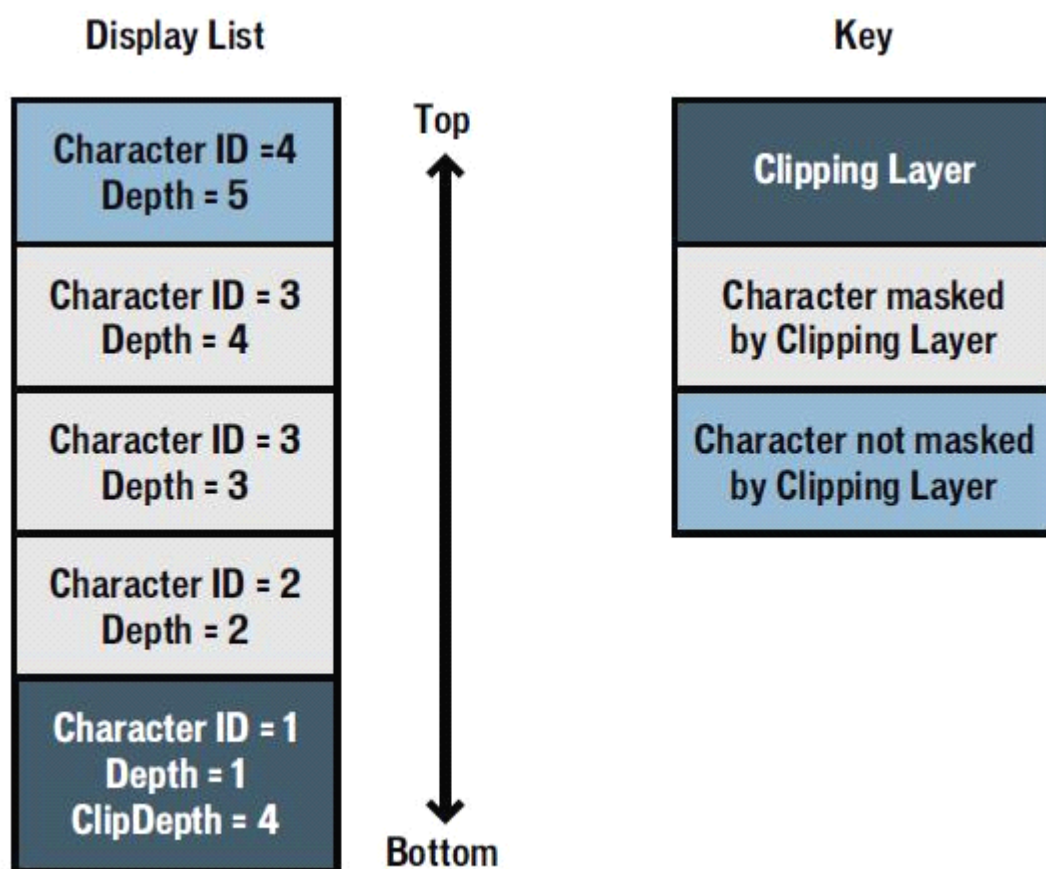
首先，定义了三个对象：一个 Shape，一个 Text 对象和一个 Sprite。这些对象给定了角色编号存储在字典里。Character 1（shape）放置到了深度1，堆栈的最底下。Character 2（text）放置了两次，一次在深度2，一次在深度4，堆栈的最上面。Character3（Sprite）放置在深度3。



剪辑层

Flash 播放器在现实列表中支持一种特殊类型的对象称作剪辑层。一个角色作为一个剪辑层放置不会显示；相当于他裁剪了（或者说遮挡了）放在上面的角色。PlaceObject2中的ClipDepth 字段指定被剪辑层遮挡的最上层深度。

例如：如果一个形状放置在1层且 ClipDepth 为4，所有1以上包括4的深度都被1层的形状遮挡了。4层以上不会被遮挡。



使用显示列表

以下是创建和显示一个动画的过程：

- 1、通过定义型标签定义每一个角色

每一个角色给一个唯一的角色编号（character ID），并添加到字典。

- 2、通过 PlaceObject2 标签将每一个角色加入到现实列表。每一个 PlaceObject2 标签都插入以下属性来指定显示的角色。

- 深度值——控制角色放置的深度。深度值低的在深度值高的角色的下面。深度值为1表

示显示在最下面。一个深度只能有一个角色。

- 转换矩阵——确定角色放置的位置，缩放，系数和旋转角度。相同的角色可以用不同的转换矩阵放置多次（角色在不同层）
- 可选的颜色变换——确定放置时角色的颜色效果。颜色效果包括透明度和颜色变换信息
- 可选的名称字符串——确定使用 `SetTarget` 动作放置时角色的名称。`SetTarget` 用于完成 `Sprite` 对象中的执行动作。
- 可选的剪辑深度（`ClipDepth`）——指定将被此角色遮挡的最上层的深度。
- 可选的比率值——控制形变角色放置的时候如何显示。比率为0，表示角色形变开始，比率为65535表示形变结束。

3、使用 `ShowFrame` 标签渲染显示列表的内容到屏幕

4、使用 `PlaceObject2` 标签修改显示列表的每一个角色。每一个 `PlaceObject2` 为给定层的对象分配一个新的转换矩阵。角色编号不用指定，因为每一个层只有一个角色。

5、使用 `ShowFrame` 标签显示角色他们的新位置。

为动画的每一帧重复4-5步骤。

提示：如果一个角色从一个帧到另一个帧没有变化，你不必理会他。

6、需要移除时使用 `RemoveObject2` 标签从显示列表中移除角色。